Concept of agent tools and memory in langchain

Author – Sambita Chakraborty

In LangChain, **Agents** act as dynamic reasoning engines that use a Large Language Model (LLM) to decide a sequence of actions. Unlike rigid chains that follow a hardcoded path, an agent evaluates user input, decides which **Tools** to use, executes them, and passes the results back into its **Memory** to plan the next step,

Ref: LangChain Deep Dive: Chains, Tools, Agents & Memory | by Amit Kharche | Medium

Workflows and agents - Docs by LangChain

---

1. The Concept of Tools

Tools are the "hands and feet" of an LLM. Because base LLMs are limited by their training data cutoffs and cannot interact with the outside world, tools extend their capabilities.

- **Definition**: A tool is a callable Python function or API wrapper with a well-defined input schema and description. [1]
- **How Agents Use Tools**: The agent reads the tool's description to understand *when* and *how* to use it. It outputs structured JSON or a function call specifying the tool name and arguments. The framework executes the function and returns the text output to the agent.
- **Common Examples**:
    - **Web Search**: Querying platforms like Google or DuckDuckGo for real-time information.
    - **Code Interpreters**: Executing Python code locally to handle complex mathematics.
    - **Database Connectors**: Fetching corporate data securely via SQL queries.
    - **MCP (Model Context Protocol)**: Seamlessly integrating standardized external tool ecosystems.
- Ref : Agents - Docs by LangChain, Tools - Docs by LangChain
- Introduction to LangChain: Build AI Agents with Python

---

2. The Concept of Memory

Memory provides the agent with continuity, ensuring it does not treat every message like an isolated, stateless query. LangChain (especially when orchestrated through LangGraph) breaks memory down by scope and time horizon,

Short-Term Memory (Thread-Scoped)

- **Purpose**: Tracks the immediate conversation history and internal reasoning steps within a single session.
- **How it works**: Managed via the agent's **State**. It records user queries, assistant responses, and the agent's internal **Scratchpad** (the step-by-step notes it takes while reasoning through multiple tool calls).
- **Context Engineering**: As the text gets too long for the LLM's context window, mechanisms like automated summarization compress older history to fit. [1, 2]

    Ref: Introduction to LangChain Agents | Aurelio AI, Context Engineering

Long-Term Memory (Persistent)

- **Purpose**: Retains user preferences, customized behavior, and domain knowledge across entirely separate conversations.
- **How it works**: Often filesystem-backed (using markdown profiles like AGENTS.md) or stored in vector databases.
- **Reflection & Adaptability**: Advanced agents use an "implicit updating" reflection loop. At the end of a session, the agent reflects on its interactions, distills your preferences (e.g., *"User prefers code examples in Python, not JavaScript"*), and updates its permanent memory.

    Ref: Memory for agents, How to Use Memory in Agent Builder, Memory - Docs by LangChain, The Anatomy of an Agent Harness, Approaches for Managing Agent Memory

---

How Tools and Memory Intersect

1. **Context Loading**: The agent boots up and loads long-term user constraints into its system prompt alongside short-term chat logs.
2. **Action Planning**: Based on memory, it recognizes it needs fresh data and calls a search **Tool**.
3. **Execution & Recording**: The tool performs the search, and the result is appended directly to the short-term memory (state) so the agent can synthesize the final answer.

Ref: How to Build AI Agents That Remember User Preferences (Without Breaking Context)

Introduction: The Evolution from Chains to Agents

To understand LangChain Agents, you must first understand the limitation of a standard Large Language Model (LLM) and basic chains.

Standard Chain: [User Input] —► [Prompt Template] —► [LLM] —► [Hardcoded Output]

Agent Loop: [User Input] —► [LLM Reasons] —► [Calls Tool] —► [Evaluates Result] —► [Final Answer]

- **What is a Chain?** A chain is a hardcoded sequence of steps. For example, taking user input, formatting a prompt, sending it to an LLM, and returning the response. The application developer pre-determines every single step.
- **What is an Agent?** An agent is a dynamic engine where the **LLM acts as a decision-maker**. You give the agent a goal and access to a suite of auxiliary utilities (Tools). The LLM evaluates the user input, decides *which* tool to use, executes it, evaluates the result, and loops until it satisfies the user's request.

---

1. Deep Dive: Agent Tools

What is a Tool?

An LLM by itself is a closed system. It cannot check today's weather, calculate complex numbers reliably, or read your internal company database. **Tools** are interfaces that allow an LLM to interact with external APIs, databases, local file systems, and code execution environments.

Anatomy of a LangChain Tool

For an agent to use a tool effectively, the tool must contain three fundamental components:

1. **The Function**: The actual code that runs (e.g., a Python function that pings a weather API).
2. **The Name**: A clear string identifier (e.g., fetch_live_weather).
3. **The Description**: A detailed text explanation of *what* the tool does and *when* to use it. **This is the most critical part.** The LLM reads this description to decide whether the tool fits the user's request.

How Agents Invoke Tools (The Reasoning Loop)

LangChain agents typically use structured paradigms like **ReAct** (Reason + Act) or Native Function Calling:

- **Reasoning**: The LLM analyzes the user prompt: *"What is the stock price of Apple?"*
- **Action**: It looks at its available tools, sees a tool described as *"Useful for getting real-time stock market data"*, and outputs a structured command (like JSON): {"action": "stock_ticker", "action_input": "AAPL"}.
- **Observation**: The LangChain framework intercepts this JSON, runs the actual Python function for stock_ticker("AAPL"), fetches the live price, and feeds it back to the LLM as an "Observation".

---

2. Deep Dive: Agent Memory

LLMs are fundamentally **stateless**. Every API call to an LLM knows absolutely nothing about the previous API calls. **Memory** is the system that preserves context across an ongoing interaction.

In modern LangChain architectures (specifically built on LangGraph), memory is split into two clear time-horizons:

A. Short-Term Memory (Thread-Scoped)

Short-term memory manages context within a single, active conversation. It is tied to a specific session identifier (often called a **Thread**).

- **The State**: It stores the list of messages (UserMessage, AIMessage, ToolMessage).
- **The Scratchpad**: When an agent takes 5 steps to solve a math problem using a calculator tool, it needs a place to write down its intermediate thoughts. The scratchpad acts as the agent's internal monologue, preventing it from repeating the same tool actions.
- **Context Truncation**: When conversations get too long, they overwhelm the LLM's maximum text capacity (context window). Short-term memory utilizes strategies like **Windowing** (keeping only the last \(N\) messages) or **Summarization** (using an LLM to compress past chats into a short paragraph).

B. Long-Term Memory (Cross-Thread Persistent)

Long-term memory remembers things *across separate days, weeks, or chat sessions*.

- **Profile Memory**: Remembers explicit user details or configurations (e.g., *"The user prefers code examples written in Python"*).
- **Semantic Memory**: Uses Vector Databases to store past interactions. If you ask a question similar to something you discussed last week, the agent can semantically search its past memories to provide context.

3. Direct Concept Comparison

Feature	Agent Tools	Agent Memory
Primary Purpose	Interacting with the external world and data systems.	Retaining context, conversational state, and preferences.
Information Flow	Pulls fresh external data **in** or pushes actions **out**.	Retains internal conversation history **across time**.
How the LLM views it	As a list of available actions with clear text descriptions.	As a backdrop of historical text or structured profiles.
Examples	Google Search, SQL Database Connector, Calculator.	Chat Message History, Vector Database profile store.

4. Architectural Diagram: